

Author: Maria Loisa Reyes

Course: AIM PostGrad Artificial Intelligence

Date: 27/9/2026

Live app: <https://graded-mini-project-14-3-next-js.vercel.app>

Repository: <https://github.com/MariaReyes99/Graded-Mini-Project-14.3-NextJS>

Reflection: Acme Widget API RAG Assistant

Week 14.3 Graded Mini Project

For this project I built the Acme Widget API Field Guide, a public RAG assistant that answers developer questions about the Acme Widget API and cites the documentation behind every answer. I started from the Week 14 Next.js RAG starter. The corpus is derived from the synthetic Acme Widget API Specification v4.2 supplied with the starter. I chose it deliberately: because I know the specification's ground truth, I can check every answer for correctness, which a large and unfamiliar corpus would not allow. I restructured the specification into eleven Markdown documents covering the overview, authentication, endpoints, rate limits, data formats, retries and timeouts, error codes, versioning and FAQs, and added two internal emails (onboarding and an API key deprecation notice) to mimic the mix of reference material and informal communication a real developer-support bot would see. My first version of the corpus was too thin, about 3,000 characters in total, so I expanded each document with the full detail from the specification, such as pagination limits, idempotency keys and retry guidance.

Embeddings use OpenAI's text-embedding-3-small model stored in Upstash Vector, and answers are generated by gpt-4o-mini. I chose gpt-4o-mini for its low cost and fast streaming on a public app, since answers depend on retrieved facts rather than deep reasoning. My original plan was 500–700-token chunks with overlap, but the Markdown headings turned out to be better boundaries than a fixed size. The seed script therefore splits on headings, so each section becomes one focused chunk (58 chunks averaging about 350 characters). A 3,000-character limit with 400 characters of overlap remains only as a safeguard for unusually long sections, and very short sections are merged into the next one. Retrieval runs as a tool call: the model decides when to call `getInformation`. To avoid general-knowledge answers, I wrote a specific tool description listing the topics the corpus covers, and a system prompt telling the model to search before answering, cite source filenames and say when the documentation does not cover a question. I retrieve the top five chunks, which gives enough context for comparative questions such as OAuth2 versus API keys without flooding the prompt with irrelevant text.

One challenge was that authentication questions sometimes retrieved endpoint content instead of the authentication sections. I stored the source filename and section heading as metadata and also prepended them to each chunk before embedding. This gave every vector explicit topic context, and authentication questions then returned the authentication chunks. The same metadata powers the Sources panel, which shows the filename, section, relevance score and retrieved text under each answer. The biggest lesson was that working locally does not mean working publicly. The environment variables had to be added to Vercel's Production environment, and my first shared link was a deployment-specific URL protected by Vercel Authentication, which sent visitors to a login page. Testing in an incognito window exposed this, and I switched to the public production domain. Because the endpoint is public, I also added simple guards: request validation, a 2,000-character question limit and a cap on the conversation history sent to the model.

The final application streams answers, uses retrieval as a tool call, displays its sources, keeps all secrets on the server, and opens with a meaningful empty state. My stretch goal was suggested-prompt chips, which appear on the empty state and in the sidebar so first-time users can start with a well-formed question.

If I continued the project, I would build a small evaluation set of question-and-answer pairs from the specification and use it to compare chunk sizes, topK values and hybrid keyword-plus-vector retrieval with measurable results rather than manual testing alone. I would also add per-user rate limiting to protect the public endpoint's API costs. Overall, this project gave me hands-on experience with the full RAG lifecycle: corpus design, chunking, embeddings, vector search, tool-based retrieval, citation rendering and production deployment.